

Horario Comienzo: 14:00 hs. Horario Fin: 17:00 hs. Cada ejercicio se debe realizar en hojas separadas.

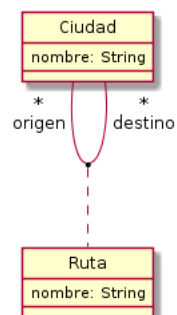
Ejercicio 1: en esta hoja Ejercicio 3: _____ hojas Ejercicio 5: _____ hojas
Ejercicio 2: _____ hojas Ejercicio 4: en esta hoja hoja

Ejercicio 1 (16 puntos): Seleccionar la opción correcta.

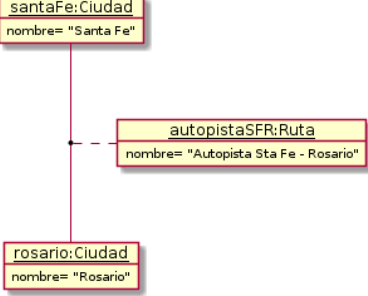
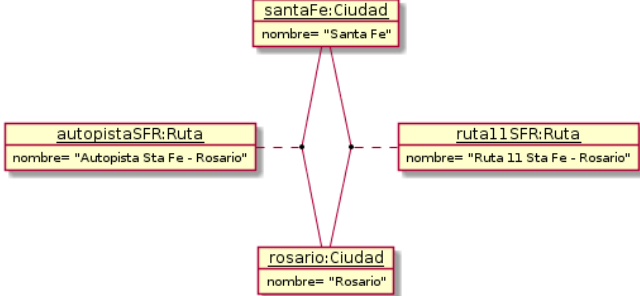
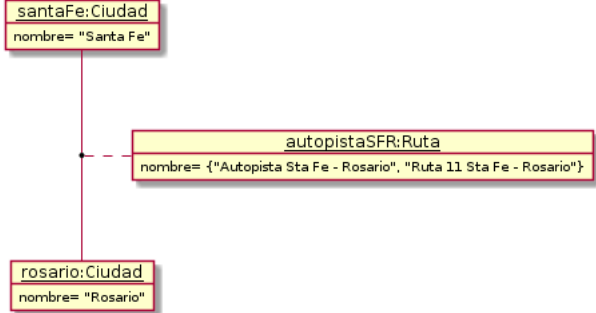
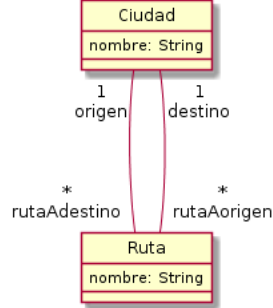
a) En un sistema bancario se desea representar que una Cuenta pertenece a una Persona o a una Empresa (no existen cuentas aisladas), pero no a ambas, por lo que el siguiente diagrama de clases es válido:

a.	<pre> classDiagram class Persona class Empresa class Cuenta Persona < -- Cuenta Empresa < -- Cuenta </pre>
b.	<pre> classDiagram class Persona class Empresa class Cuenta Persona < -- Cuenta Empresa < -- Cuenta </pre>
c.	<pre> classDiagram class Persona class Empresa class Cuenta Persona < -- Cuenta Empresa < -- Cuenta </pre>
d.	<pre> classDiagram class Persona class Empresa class Cuenta Persona < -- Cuenta Empresa < -- Cuenta </pre>
e.	<pre> classDiagram class Persona class Empresa class Cuenta Persona < -- Cuenta Empresa < -- Cuenta </pre>
f.	Ninguna de las opciones previas es válida

b) Considere el siguiente diagrama de clase:



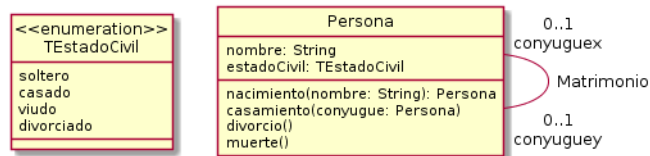
, el cual es instanciado por:

	 Si se desea representar otra ruta entre santaFe y rosario:
a.	Se instancia nuevamente Ruta vinculando a las ciudades santaFe y rosario, siendo el siguiente diagrama un diagrama de objetos válido: 
b.	Se modifica la instancia autopistaSFR, incorporando el nuevo nombre de la ruta, siendo el siguiente diagrama un diagrama de objetos válido: 
c.	El modelo existente no permite representar múltiples links entre los mismos objetos, podría modificar el diagrama de la siguiente manera:  De esta manera, por cada link entre ciudad se instanciaría Ruta con las características adecuadas.
d.	El modelo existente no permite representar múltiples links entre los mismos objetos, podría modificar el diagrama de la siguiente manera:

	De esta manera, por cada link entre ciudad se instanciaría Ruta con las características adecuadas.
e.	El modelo existente no permite representar múltiples links entre los mismos objetos, podría modificar el diagrama de la siguiente manera: De esta manera, por cada link entre ciudad se instanciaría Ruta con las características adecuadas.
f.	Ninguna de las opciones previas es válida

c) Dada la relación de composición	
a.	para representar que un Auto está compuesto por muchas AutoPartes y que tengo AutoPartes que pertenecen a un Auto y otras no (no están vinculadas con Auto, por ejemplo, serán usadas como repuestos), es posible realizar el siguiente diagrama:
b.	para representar que un Auto está compuesto por muchas AutoPartes y que tengo AutoPartes que pertenecen a un Auto y no pueden ser objetos aislados, es posible realizar el siguiente diagrama:
c.	para representar que un Auto está compuesto por muchas AutoPartes y que tengo AutoPartes que pertenecen a cero, uno, o más Autos, es posible realizar el siguiente diagrama:
d.	Las alternativas de modelados a, b, c son válidas para cada uno de los problemas planteados.
e.	Ninguna de las opciones previas es válida.

d) Dado el siguiente modelo:



context Persona::nacimiento(nombre: String): Persona

post: result.ocllsNew() and result.nombre = nombre and result.estadoCivil = TEstadoCivil::soltero

context Persona::casamiento(conyugue:Persona)

pre: self.estadoCivil <> TEstadoCivil::casado and conyugue.estadoCivil <> TEstadoCivil::casado

post: self.conyuguey = conyugue and self.estadoCivil = TEstadoCivil::casado and conyugue.estadoCivil = TEstadoCivil::casado

context Persona::divorcio()

pre: self.estadoCivil = TEstadoCivil::casado

post: self.estadoCivil= TEstadoCivil::divorciado and

(self.conyuguey@pre->notEmpty() implies self.conyuguey@pre.estadoCivil=TEstadoCivil::divorciado and

self.conyuguey->isEmpty()) and

(self.conyuguey@pre->notEmpty() implies self.conyuguey@pre.estadoCivil=TEstadoCivil::divorciado and

self.conyuguey->isEmpty())

context Persona::muerte()

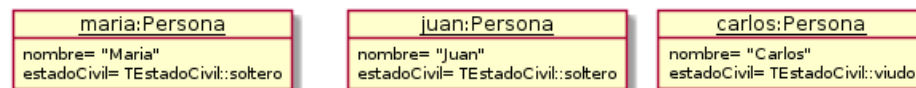
post noexistemas: Persona.allInstances()->excludes(self@pre)

post viudo:

(self@pre.conyuguey@pre->notEmpty() implies self@pre.conyuguey@pre.estadoCivil=TEstadoCivil::viudo) and

(self@pre.conyuguey@pre->notEmpty() implies self@pre.conyuguey@pre.estadoCivil=TEstadoCivil::viudo)

y teniendo el siguiente modelo de objeto:



Es posible ejecutar la siguiente secuencia de operaciones

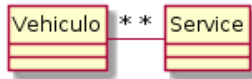
maria.casamiento(juan); maria.divorcio(juan); maria.casamiento(carlos); carlos.muerte()

siendo el modelo de objeto resultante

a.	maria:Persona nombre= "Maria" estadoCivil= TEstadoCivil::divorciado juan:Persona nombre= "Juan" estadoCivil= TEstadoCivil::divorciado
b.	maria:Persona nombre= "Maria" estadoCivil= TEstadoCivil::viudo juan:Persona nombre= "Juan" estadoCivil= TEstadoCivil::divorciado
c.	maria:Persona nombre= "Maria" estadoCivil= TEstadoCivil::viudo juan:Persona nombre= "Juan" estadoCivil= TEstadoCivil::divorciado carlos:Persona nombre= "Carlos" estadoCivil= TEstadoCivil::difunto :Matrimonio
d.	maria:Persona nombre= "Maria" estadoCivil= TEstadoCivil::viudo juan:Persona nombre= "Juan" estadoCivil= TEstadoCivil::divorciado :Matrimonio
e.	Ninguna de las opciones previas es válida.

Ejercicio 2 (16 puntos):

- a) Suponga que tiene que representar las siguientes dos reglas de negocio:
rn1) A todo vehículo se le puede realizar muchos servicios.
rn2) Todo service se puede aplicar a muchos vehículos.
Entonces, propone el siguiente diagrama:



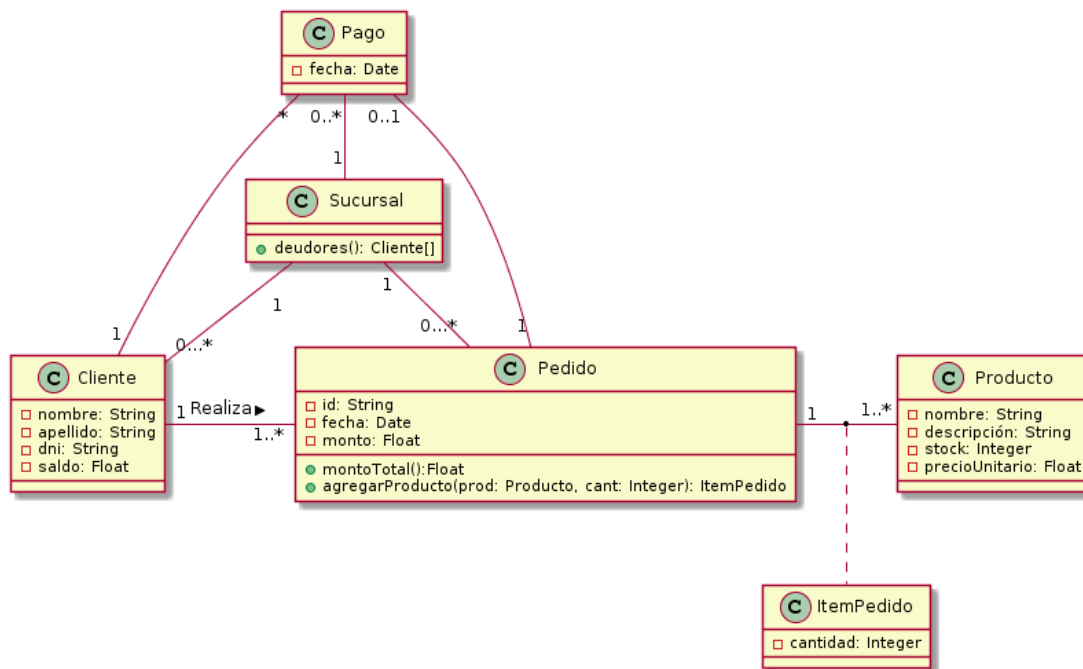
Luego, se añade la siguiente regla de negocio:

rn3) cada service realizado a un vehiculo es registrado, indicando kilometraje del vehículo y repuestos cambiados.

Modifique el diagrama para representar las 3 reglas de negocio enunciadas.

- b) Analice si el modelo propuesto por usted en el ítem a permite que un vehículo realice 1 o más veces el mismo service. Justifique la respuesta realizando un diagrama de objetos que ilustre si es factible o no. En caso de que no lo permita, proponga una modificación (haciendo uso de UML u OCL) para que un vehículo pueda realizar múltiples veces el mismo service.
- c) Modifique el modelo propuesto en los ítems previos para representar las siguientes reglas de negocio (emplee UML y/o OCL):
rn4) un mecánico, es una persona que realiza un service a un vehículo.
rn5) un cliente, es una persona propietaria de un vehículo.
- d) Modifique el modelo propuesto en los ítems previos (emplee UML y/o OCL) para que represente las siguientes regla de negocio:
rn6) una persona puede ser mecánico en un service, pero propietario de un vehículo y cliente de otro service.
rn7) una persona no puede ser mecánico y cliente del mismo service.

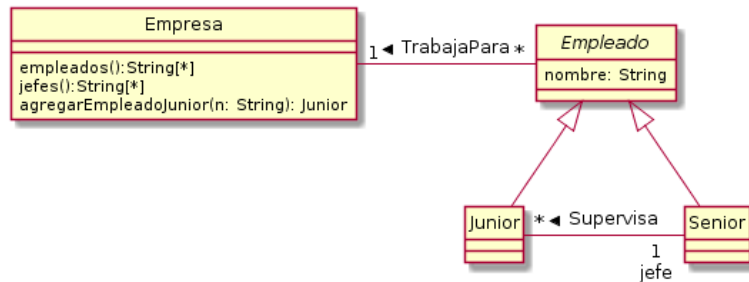
Ejercicio 3 (21 puntos): Dado el modelo definido, especifique las restricciones OCL que se solicitan:



- a) Invariante que asegura que el saldo de un cliente es menor o igual a la sumatoria de los montos de los pedidos que realizó el cliente.
- b) Invariante que asegura que dentro de una Sucursal, no puede haber dos clientes con el mismo dni.
- c) Restricción que exige que el pago de un pedido debe realizarse en la misma sucursal en la que fue realizado el pedido.

- d) Especificar la operación de consulta montoTotal() de la clase Pedido, la cual retorna la sumatoria del producto del precioUnitario de los ítems pedidos por su cantidad.
- e) Especificar la operación de consulta deudores() de la clase Sucursal, la cual retorna aquellos clientes que tienen pedidos en la sucursal que no han sido pagados (no están asociados a Pago).
- f) Especificar la precondition de la operación agregarProducto(prod: Producto, cant: Integer) de la clase Pedido, teniendo en cuenta:
 - el producto solo puede agregarse si no existe ya en el pedido;
 - el producto solo puede agregarse si existe stock suficiente.
- g) Especificar la poscondition de la operación agregarProducto(prod: Producto, cant: Integer) de la clase Pedido, como resultado el pedido tiene el producto agregado con la cantidad especificada.

Ejercicio 4 (16 puntos): Dado el siguiente modelo, seleccionar la opción correcta.



a) La operación de consulta jefes (empleados que son Senior y supervisan al menos un Junior) se puede especificar en OCL mediante:	
a.	context Empresa::jefes(): Set(String) body: self.TrabajaPara->collect(nombre)->asSet()
b.	context Empresa::jefes(): Set(String) body: self.empleado.nombre->select(e e.ocllsTypeOf(Senior) and e.junior->notEmpty())->asSet()
c.	context Empresa::jefes(): Set(String) body: self.empleado->select(e e.ocllsTypeOf(Senior) and e.ocllsType(Senior).junior->notEmpty())->collect(nombre)
d.	context Empresa::jefes(): Set(String) body: self.empleado->select(e e.ocllsTypeOf(Senior) and e. ocllsType(Senior).junior->notEmpty()).nombre->asSet()
e.	context Empresa::jefes(): Set(String) body: self.empleado->select(e e.ocllsTypeOf(Senior) and e.junior->notEmpty())->collect(nombre)
f.	Ninguna de las opciones previas es válida.

b) La operación de agregarEmpleadoJunior instancia Junior con nombre igual a n y ese empleado TrabajaPara la empresa que agrega el empleado. Este comportamiento se especifica en OCL mediante:	
a.	context Empresa::agregarEmpleadoJunior(n: String): Junior post: result.ocllsNew() and n = result.nombre and self.empleado@pre->including(result) = self.empleado
b.	context Empresa::agregarEmpleadoJunior(n: String): Junior post: result.ocllsNew() and result.nombre = n and self.empleado->includes(result)
c.	context Empresa::agregarEmpleadoJunior(n: String): Junior post: result.ocllsNew() and n = result.nombre and self.junior@pre->including(result) = self.junior
d.	context Empresa::agregarEmpleadoJunior(n: String): Junior post: result.ocllsNew() and result.nombre = n and self.junior->includes(result)
e.	Ninguna de las opciones previas es válida.

c) La siguiente invariante OCL siempre se cumple:	
a.	context Junior inv: self.jefe.empresa->size() = 1
b.	context Junior inv: self.jefe.empresa->size() > 1
c.	context Junior inv: self.jefe.TrabajaPara->size() = 1
d.	context Junior inv: self.jefe.empleado.empresa->size() > 1
e.	context Junior inv: self.jefe.empleado.empresa->size() = 1
f.	Ninguna de las opciones previas es válida.

d) Definiendo la especialización de Empleado como {disjoint, complete}:	
a.	context Empleado inv: self.ocllsTypeOf(Junior) and self.ocllsTypeOf(Senior)
b.	context Empleado inv: self.ocllsTypeOf(Junior) implies self.ocllsTypeOf(Senior)
c.	context Empleado inv: self.ocllsTypeOf(Junior) implies (not self.ocllsTypeOf(Senior)) inv: (not self.ocllsTypeOf(Junior)) implies self.ocllsTypeOf(Senior)
d.	context Empleado inv: (self.ocllsTypeOf(Junior) or self.ocllsTypeOf(Senior)) and self.ocllsTypeOf(Empleado)
e.	Ninguna de las opciones previas es válida.

Ejercicio 5 (31 puntos): Dado el siguiente enunciado, realice un modelo de análisis que represente estáticamente al sistema. El modelo debe incluir un **diagrama de clases UML** que incorpore toda la información brindada y los conceptos que conoce (clases, relaciones, roles, multiplicidad, atributos, etc.).

Se desea desarrollar un sistema para dar soporte a un servicio de alquiler de vehículos ofrecido por una compañía en una gran ciudad, que es visitada por miles de turistas cada año. El objetivo es poder manejar toda la información de reservas, alquiler y devolución de los rodados disponibles, así como también de los clientes.

El sistema debería permitir a los clientes consultar el catálogo de vehículos disponibles, filtrarlos por tipo y por rango de precios, reservarlos y pagarlos.

De cada cliente se desea almacenar su DNI, nombre, apellido, dirección y teléfono celular. Además, es importante mantener también un código de cliente, que es utilizado en el proceso de alquiler mencionado.

Las reservas no tienen costo, pero pueden realizarse entregas. Una reserva la realiza un único cliente y sólo puede involucrar un vehículo. El alquiler de un vehículo tiene un precio asociado, que es particular para cada vehículo. Dicho precio puede ser modificado por otro valor o a través de un porcentaje de incremento. Cuando se alquila un vehículo, es importante registrar la fecha de inicio y final del alquiler, el precio total del alquiler (es calculado antes cada pago considerando el vehículo, período de reserva y medio de pago), los litros de combustible que dispone el vehículo al momento de realizar el alquiler. Las reservas tienen una fecha de reserva y una fecha de caducidad. Una reserva puede ser "cancelada", "suspendida temporalmente" o "pagada" (entrega). Un alquiler implica que el estado del vehículo es "alquilado". Sin embargo, un vehículo también puede estar en estado "reservado" o "en reparación". Los vehículos son de 2 tipos distintos: vehículos para uso familiar y vehículos para uso con fines turísticos, que tienen una capacidad superior de pasajeros. La reserva de este último tipo de vehículos incluye un costo adicional porque este tipo de vehículos tiene incluido también un chofer asociado y un costo adicional de impuestos. Los medios de pago disponibles para los alquileres y reservas son los siguientes: tarjeta de crédito (número y código de seguridad) y transferencia bancaria (se deben registrar banco, cuenta y número de transferencia). Cada vehículo tiene un dominio asociado (patente), marca, modelo, color, número de motor y cantidad máxima de pasajeros. Además, cada vehículo está asignado a una plataforma dentro del parque de estacionamiento de la empresa, para poder ubicarlo fácilmente. Las plataformas tienen un número de identificación.

Una vez que los clientes realizan el pago, deben acercarse personalmente a retirar el vehículo. Allí se les entrega una tarjeta de alquiler que contiene los datos más importantes del alquiler y la documentación del vehículo. Si el vehículo alquilado es del tipo uso con fines turísticos, el chofer designado es quien recibe al cliente y lo acompaña hasta el vehículo. Este tipo de vehículos es siempre manejado por el chofer asignado, por cuestiones legales. El sistema debe registrar para los choferes todos los datos que almacena para un cliente, pero además debe incorporar otros datos como el número de CUIL, fecha de ingreso a la empresa y otra documentación relacionada (que es también es gestionada por el sistema). El dueño de la empresa pretende que la herramienta le brinde múltiples reportes de consulta, entre los cuales se pueden mencionar el listado de vehículos por cada tipo, los alquileres, las reservas y los pagos recibidos por fecha.